



DuelMaze Programming Guide

Objectives:

This guide aims to help you with the syntax of C++, and how to program the components being used in the project. These components include the servos, I2C LCD, and the Arduino itself.

How to use this guide:

The main objective of this guide is to help the members understand what they are going to be working with, and to reduce the time spent on teaching, and utilize it to actually work on the project. Whenever you are stuck on a problem, use this guide first to try and understand the correct way of doing things, and THEN ask one of the UC's, or your Team Members.

1. C++ Syntax:

The language used in Arduino programming is CPP, albeit a more limited version of it. For the scope of this project, we do not need very complicated C++ knowledge. Read the example below to understand how to do basic programming tasks in C++.

1.1 Variables and Data Types:

Variables are by far one of the most important parts of programming. Make sure to learn how to use Data Types and variables, because they are fundamental to being able to participate.

```
int a, b; //This is how you initialize variables
a = 10; b = 16;
string myName = "Rajesh"; //init, and also declare together
double x = 3/5; //"double" is used when you need the decimals
```

Notice how in C++, unlike Python, the variables' data types need to be declared. Also, the variables need not be assigned a value while declaring them, that can be done later as well, as in the case of variables 'a' and 'b' in our case here. However, the variable 'myName' has been declared and given a value at the same time as you can see.

The data type **int** is used when we want to store an integer(lol), and the **string** data type is used when we need to store letters, words, etc. the **double** type, called "floating point" in other languages, is used to store numbers with decimal points in them.

These data types are very important, and will be used during the course of the project.

1.2 Flow of Control:

Flow of control is another important aspect of programming, and is almost always the backbone of the program. There are mainly 2 types of flow, namely decisions(if, else) and loops(while, for).

1.2.1 IF, ELSE:

If and else statements are used when there is a need to execute only parts of the program, depending on the condition specified. An example is given below:



```
if (number > 0) {  
    // Print that number is positive here  
}  
// Else if statement checks if the number is negative  
else if (number < 0) {  
    // Print that the number is negative  
}  
// Else statement is executed if the number is neither positive nor  
// negative (i.e., it's zero)  
else {  
    // Say that the number is zero  
}
```

In this case, we have some variable, **number**. It's not necessary to know where this variable comes from, since this is just an example. The code is self explanatory, because of the comments. Now, let's move on to Looping.

1.2.2 Looping:

Loops are similar to if and else statements, in the sense that they also check for a condition. The only difference is that instead of executing that particular block of code just once, they do it again and again, until that condition becomes false. There are mainly 3 types of loops in C++, but we'll only be looking at 2, which are used in all languages. They are: **while loops** and **for loops**.

1.2.2.1 For loops:

For loops are used, when we know exactly how many times we want to repeat a block of code, or when we want to iterate through a list (this will be discussed later). For example, here, this block of code prints numbers all the way from 1 to 5.

```
// Simple for loop to print numbers from 1 to 5  
for (int i = 1; i <= 5; i++) {  
    cout<<i<<endl;  
}
```

In the code here, notice the syntax of the for loop. First, we declare a **control** variable, called **i**, and we give it a value of **1**. Then, we tell the loop the condition. In this case, the condition is that **i** must be less than or equal to 5. The last part is **incrementing** the value of **i**. So, in the first iteration (iteration means the same as repetition), the value of **i** is 1. In the second iteration, it is 2, and so on, until it becomes 5. When the value of **i** becomes 6, the condition we specified, becomes false, and the loop stops. This is how the output looks.



```
1  
2  
3  
4  
5
```

1.2.2.2 While Loops:

While loops are similar to For Loops, in the sense that the code executes based on a control variable, say **i**. The loop executes till the condition specified is true, and this can be ANY condition, unlike in a for loop, where it only executes based on value. Unlike a for loop, the value of the control variable in this case does not need to change every iteration. This will be more useful in complex programs. Here's the same program we saw earlier, but written using a while loop.

```
// Simple while loop to print numbers from 1 to 5  
int i = 1;  
while (i <= 5) {  
    cout<<i<<endl;  
    i++; // Increment i to avoid an infinite loop
```

Now, notice how the syntax is different from a for loop. We declare a variable, **i**, but outside the loop, and then we only check for the condition when writing the loop. Now, another important thing to notice is that we change(in this case, we are incrementing) the value of **i**, INSIDE the body of the loop. It is VERY important to make sure that this value changes, or else the loop will become infinite, which we don't want.

Alright! You've just learnt the basics of C++ programming! There are many more topics left, such as arrays, etc. But they'll be taught to you during the sessions. Now, let's move on to the libraries that we'll be using in arduino programming, and the structure of an arduino program.

2. Arduino Programming:

This second section will cover the Arduino program structure, and the 2 libraries that we'll be using, which is the LCD-I2C, and the Servo.

2.1 Arduino Program Structure:

The basics of arduino programming were taught to you, but let's recap:

```
void setup(){  
    // Runs only ONCE  
}  
  
void loop(){  
    //Keeps running again and again.  
    //This is where the main program goes.  
}
```



2.2 Servos:

Most of you have already worked with servos in our previous makerspace sessions. In this part, let's go over how to actually turn a servo.

```
#include <Servo.h> //This is the servo library, which is downloaded.

Servo myServo; //We are declaring a variable, and telling the arduino
               // that this is a servo.

int pos; //Declaring a variable to store the servos position.
         // Recall how to declare a variable and what type this is.
void setup(){
    myServo.attach(9);
    //Tell the arduino that this is the pin on which you have attached
    // the servo
}

void loop(){
    for (pos = 0; pos <= 180; pos += 1) { // goes from 0 degrees to 180 degrees
        // in steps of 1 degree
        myservo.write(pos);              // tell servo to go to position in variable 'pos'
        delay(15);                       // waits 15ms for the servo to reach the position
    }
    for (pos = 180; pos >= 0; pos -= 1) { // goes from 180 degrees to 0 degrees
        myservo.write(pos);              // tell servo to go to position in variable 'pos'
        delay(15);                       // waits 15ms for the servo to reach the position
    }
}
```

You all must have already written this program in your makers sessions. In case you ever forget how to use a servo, please refer to this code snippet, and see what you missed.

2.3 LCD I2C:

The LCD I2C has already been discussed in the session. This is an example code, please refer to it in case you need to use the LCD.



```
#include LiquidCrystal_I2C.h //Import the LCD Library
#include Wire.h //Needed for the I2C
//the first parameter is the I2C address
//the second parameter is how many rows are on your screen
//the third parameter is how many columns are on your screen
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  //initialize lcd screen
  lcd.init();
  // turn on the backlight
  lcd.backlight();
}

void loop() {
  //wait for a second
  delay(1000)
  // tell the screen to write on the top row
  lcd.setCursor(0,0);
  // tell the screen to write "hello, from" on the top row
  lcd.print("Hello, From");
  // tell the screen to write on the bottom row
  lcd.setCursor(0,1);
  // tell the screen to write "Rajesh" on the bottom row
  // you can change whats in the quotes to be what you want it to be!
  lcd.print("Rajesh");
}
```